DeepFake Detection - Documentation

Pietro De Angeli

December 24, 2025

Contents

1 Introduction

This project, part of the "Signal, Image and Video Processing" course, implements the paper *"Efficient Temporally-Aware DeepFake Detection using H.264 Motion Vectors"*. The goal is to distinguish between DeepFake and real videos using **motion vectors** (MVs), a lightweight alternative to **Optical Flow** (OF).

MVs can be extracted directly from the H.264 video encoding (if supported), without further computation. Only motion vectors inside face regions are used, which are detected using the pretrained **YuNet** face detector (a lighter alternative to MTCNN used in the original paper).

The architecture consists of two identical MobileNetV3 networks, slightly modified, where one processes RGB images and the other MV+IM tensors. Their outputs are then combined to produce a final binary classification.

The paper criticizes traditional methods for only using per-frame predictions without considering temporal consistency. This solution leverages such temporal information by comparing how content moves across frames.

Unlike OF, which operates at the pixel level, MVs describe how macroblocks (e.g., 16×16 pixels) move across frames. While less precise, MVs are significantly faster to obtain. Their goal was to find a lighter version of optical flow that could be used online and on low-power devices; inference is efficient and bottlenecked mostly by face detection.

2 Project Structure

```
|- dataset/
   |- video_1/
   |- ...
   |- video_N/
|FF++/
|- models/
|- src/
   |- classifier/
   |- data_preprocessing/
   |- tools/
   |- UI
|- test/
```

2.1 FF++

The dataset used is the original FaceForensics++ dataset. It was chosen because the paper suggests it.

The subset consists of two directories:

- *real*: containing 1000 original videos;
- *fake*: containing 1000 DeepFake videos.

2.2 dataset

The dataset described above has been preprocessed and restructured into subdirectories, as explained in Section 2.4.2. Each video is stored in its own directory containing the following files:

- `meta.json`: contains the original path of the video, the label (*real* or *fake*), and the number of frames extracted. Typically 100, although sometimes fewer.
- `tensors.pt`: contains the stack of `motion_vector_x`, `motion_vector_y`, and `information_mask` for each frame.
- `faces/`: contains the cropped images of the detected faces.

A global file `manifest.json` is also created, containing two objects: `train` and `test`. Each entry specifies the directory of the video and its corresponding label. The split ratio is 80% for training and 20% for testing.

The manifest file is generated by a script described in Section 2.4.2.

2.3 models

- **YuNet**: a pretrained face detector that outputs bounding boxes and confidence scores. It is the bottleneck of our system and is used to extract faces from 100 random and unique frames of each video.
- **Classifier**: based on **MobileNetV3-Small** with pretrained weights. It implements a two-stream architecture that processes both RGB frames and MV+IM features, outputting the prediction of whether a video is fake.

  The paper suggests modifying the base network by adapting the **input** (3 channels, 224x224 images) and the **output** (binary classification). In addition, we introduced a parameterized weighted average to combine the two streams. Unlike our implementation, the original paper uses a fixed, non-learnable averaging.

2.4 src

This directory contains the Python source code, organized into submodules.

2.4.1 classifier

Contains the classifier implementation (Section 2.3).

- `MobileNetV3.py`: downloads the pretrained model and adds additional layers.
- `network.py`: defines the two-stream architecture.
- `training.py`: training procedure (Section 4).
- `evaluation.py`: evaluates model accuracy.

2.4.2 data_preprocessing

Handles preprocessing, including train/ split.

`create_dataset.py` processes each video (pipeline in Section 3), creating the structured dataset. `split_dataset()` generates `manifest.json`, splitting into train (80%) and test (20%).

2.4.3 tools

Contains utility modules:

- `face_detection.py`
- `motion_vectors.py`
- `feature_computation.py`
- `tools.py`

2.5 Tools

Provides helper functions:

- `get_dir_videos()`: returns list of all `.mp4` files in a directory.
- `pipeline()`: runs the entire inference pipeline.

  Explained in Section 3.

2.6 `UI`

Contains a simple user interface:

- **Upload Page**: drag-and-drop or browse a video. Clicking "Detect" starts the analysis.
- **Progress Page**: shows a waiting screen during computation.
- **Result Page**: shows final results, execution time for each step, and a button to open the temporary directory with extracted faces.

2.7 `test`

Contains unit tests for the `src` functions (`pytest`). Checks:

- Face detection: initialization of YuNet, square box boundaries, and extraction of frames with valid `Face` objects.
- Motion vectors: correct handling of I-frames (zero maps), filling of macroblocks in P-frames, and `is_in_face()` logic.
- Feature computation: standardization of motion vectors only on inter-coded pixels, masking of intra-coded ones, and stacking into tensors with the expected shape.
- Video-level features: correct stacking of multiple frames into a single tensor with consistent dimensions.

3 Pipeline

The pipeline is described in the following sections. A complete implementation of the inference pipeline is available in the file `tools.py` (see Section 2.4.3), through the function `pipeline()`.

3.1 Face Detection

The original paper suggests using MTCNN, but YuNet was chosen instead for its efficiency. Two main classes are defined:

- `FaceBox`: represents a square bounding box (x, y, side).
- `Face`: represents a bounding box and the corresponding 224×224 BGR face image.

The function `initialize_detector()` loads the YuNet model. The function `face_frame_extractor()` returns the largest detected face (if any), cropping and resizing it to 224×224 pixels while centering the bounding box.

Frames can be extracted using two different methods:

- `random_frame_extractor()`: samples random timestamps, decodes a short sequence of frames, and selects one.
- `unique_frame_extractor()`: uses reservoir sampling to guarantee uniform coverage over the entire video.

The wrapper `extract_frames_with_faces()` selects between the two approaches based on the `unique_frames` flag. These functions return up to 100 tuples of the form (frame, face). To enable motion vector extraction, the video is opened with the following option:

```
stream.codec_context.options = {"flags2": "+export_mvs"}
```

3.2 Motion Vector Extraction

The class `MotionVector` stores the following attributes:

- `dst_x`, `dst_y`: current block location
- `motion_x`, `motion_y`: displacement values
- `motion_scale`: scaling factor

The method `is_in_face()` checks whether a motion vector falls inside a detected face region.

The function `extract_motion_vectors_and_im()` processes each frame–face pair and returns a list of tuples: (MV_x, MV_y, IM). It handles different frame types:

- I-frames (all-zero motion vectors),
- P/B-frames containing valid motion vectors,
- frames where motion vectors may be missing.

Each motion vector is mapped to macroblocks, and intra-coded blocks are identified using the **Information Mask (IM)**: 0 = inter-coded (with MV), 1 = intra-coded. All outputs are resized to 224×224 using appropriate interpolation.

3.3 Feature Computation

The function `compute_features_frame()` extracts per-frame features as follows:

- Normalizes motion vectors over inter-coded regions (IM == 0).
- Stacks MV_x, MV_y, and IM into a 3-channel feature map.

Then, the function `compute_features_video_tensor()` aggregates the per-frame features into a single video tensor with shape (frames, height, width, 3).

3.4 Classification

Finally, the faces extracted during the Face Detection phase and the tensors containing motion vectors are fed into the network. The classifier is restored from the `models` directory (see Section 2.3), and all inputs are passed through the network to produce the final prediction.

4 Training the Model

The training was performed on 70% of the entire dataset, 15% for the validation and 15% for the test. The setup consisted of 10 epochs; however, due to the early stopping strategy (implemented using K-fold validation), the training stopped after 9 epochs.

Two training experiments were carried out:

1. **Without pretraining:** starting from random weights and without augmentation, the model achieved an accuracy of approximately 41%.
2. **With pretraining:** starting from a pretrained model, data augmentation was applied. For the first 5 epochs, only the newly added layers were trained while keeping the backbone frozen. Afterwards, all weights were unfrozen and fine-tuned. This approach yielded an accuracy of approximately 64%.

4.1 Augmentation

As suggested in the reference paper, the training set was augmented. The augmentation was applied *on the fly*, i.e., only during training and under a certain probability. The library `albumentations` was used to apply the following transformations:

- **RGB distortion**
- **Random brightness and contrast adjustments**
- **Horizontal flips**: also the motion vectors are flipped

It has not been used classical techniques such as Gaussian noise, Gaussian blur$\cdots$ because the paper states they reduce the accuracy.

4.2 Evaluation

The evaluation was performed on the test set (15%) and it reached the accuracy of 71%, the same the original paper reached.

5 Tools & Technologies

5.1 Conda

To set up the project, a dedicated `conda` environment was created with `Python==3.10`. A `setup.py` file was provided to automatically install the required dependencies by running `pip install .` from the main directory.

5.2 Libraries

The main external libraries used are:

- `av` (15.0.0): used to extract motion vectors from videos.
- `albumentations`: applied during training for data augmentation, helping mitigate overfitting.
- `opencv`: used for image and video processing.
- `pytest`: used for unit testing.
- `torch`: used for training the neural network and for the face detection module.

5.3 RunPod

Since training on a local machine was too slow, a cloud platform was used. Specifically, training was performed on a *Pod* hosted on RunPod. Using an `ssh` connection, the preprocessed dataset and the source code (from GitHub) were uploaded, the environment was set up via the `setup.py` file, and the training was executed remotely.

The Pod image came with PyTorch and CUDA pre-installed, and the training was run on an NVIDIA RTX 4090 GPU.

6 Results

The main objective of the project was to reproduce the performance reported in the paper while mitigating the bottleneck in the face detection phase. The same level of accuracy was achieved using a lightweight model, YuNet, which simplifies the pipeline.

7 Future Works

The main limitation concerns the use of YuNet. Although it is more efficient than MTCNN, it remains the bottleneck of the pipeline. Future work could therefore explore even lighter models, or potentially solutions that avoid AI-based face detection altogether.